

An Experimental Study of Load Balancing on Amoeba

Weiping Zhu C.F. Steketee
School of Computer and Information Science
University of South Australia
Adelaide, Australia SA5095

Abstract

This paper presents the results of an experimental study of load balancing using job initiation and process migration, carried out on Amoeba. The results indicate the need for a load balancing facility in a distributed system to improve system performance, e.g., the average response time of processes. A number of load balancing algorithms, including the bidding and neighbouring algorithms, have been studied in this work. A comparison between these algorithms under various conditions is presented, which indicates that in a system with 10 - 20 computers a centralized algorithm outperforms a distributed one and job initiation plays an important role in a load balancing scheme. We also point out some requirements for an operating system in order to support an efficient load balancing facility, on the basis of our experience. We conclude with a summary of our experiences and suggestions for further work.

1 Introduction

A distributed computer system with tens or hundreds of computers connected by high-speed networks has many advantages over a system that has the same computers, but stand-alone. One of the most important advantages of a distributed system is its provision for efficient resource sharing. This implies that under the same conditions a distributed system should provide much better service than a traditional system in terms of performance and reliability. In order to achieve the performance potential of a distributed system, a dynamic load balancing facility is required. This facility monitors load variation to detect load imbalance, and then takes action to balance the load. Placing, replacing (redistributing) and replicating some objects in a system are the possible actions which can be adopted by a load balancing facility. In this paper, we present the results of an ex-

perimental study of load balancing, which only takes processes into account.

A considerable number of research projects on load balancing has been carried out over many years due to the potential performance gain from this service. Most of the research has focused on load balancing algorithms and load measurement. The aim is to find general effective load balancing algorithms with overhead as low as possible. The methods used in the past range from analytic study plus simulation to implementation plus testing. Due to the difficulty of implementing a load balancing facility in a distributed system, the majority of studies fall into simulation. Different assumptions and models used by these researches lead to quite different results which divides the researchers into different camps. For instance, some are in favor of a non-preemptive strategy [6], others prefer the preemptive approach [8], [9].

On the other hand, only a few projects, e.g. MOSIX [1], Utopia [15], Mach [10], were involved in designing and implementing a load balancing facility into distributed systems. Due to lack of practical experience, a number of questions related to the design of a load balancing facility have not been answered, for instance, how to measure the load of a computer, the frequency used sampling load data, etc. In view of the shortage of experimental studies, we decided to conduct a series of tests of various load balancing algorithms on a real system and compare their performance under various conditions. The comparison focuses on the performance and scalability. This study also aims to verify previous results received from simulation studies. The Amoeba system [12] has been selected to carry out these experiments because of its ready availability.

The results presented in this paper by executing synthetic workload benchmarks with various load balancing strategies indicate the necessity of a load balancing facility for a distributed system for optimum performance. They also reveal that based on process migration, a preemptive load balancing strategy can

further improve the average response time of processes only when the average load of a system exceeds a moderate level, which matches the result reported by Eager *et al.* based on analytic study [6]. Our result also shows that within a system that consists of 10 – 20 computers a centralized load balancing strategy outperforms a distributed one in both performance and cost, agreeing with Zhou's result obtained from trace-driven simulation [14].

This paper is organized in four sections: in the next section, we outline the structure of the load balancing facility embedded into the Amoeba system. Section 3 is devoted to the experimental study, discussing the load balancing algorithms used in this study and presenting their performance under various load conditions. The last section is devoted to the conclusion.

2 Load Balancing Facility

Amoeba was not designed and implemented with dynamic load balancing in mind because the designers advocate the processor-pool model [12] which assumes the number of processors is more than the number of processes. However, current computing environments typically contain autonomous workstations connected through networks, which requires load balancing for its performance. Thus, we configured a workstation environment for the experiment and retrofitted the load balancing facility into Amoeba.

2.1 The Structure of Load Balancing Facility

In order to support both preemptive and non-preemptive load balancing strategies, we implemented the load balancing facility as two sort of processes: load balancers and process migration servers. A load balancer makes decisions on *when to move which process to where*, while, a migration server carries out the decision made by a load balancer to move processes between computers. Therefore, the load balancer plays the policy role in this facility. On the other hand, a process migration server supplies the mechanism in the load balancing facility.

This separation between policy and mechanism offers a flexible structure for our experimental study of various load balancing algorithms. The process migration server provides the same service, regardless of the differences between policies. We first implemented the process migration server in the system [11]. The server has a standard interface, based on RPC, to link load

balancers. The decision to implement process migration in a distinct server minimizes the additional logic required in the kernel.

2.2 Process Migration on Amoeba

An Amoeba process may have multiple threads, each of which is an object that may be scheduled by the kernel. During a process migration, all threads of the process must be suspended and moved from the source to the destination. The execution state of a process is marshalled into messages and sent to the destination to reconstitute.

Besides the execution state, the memory space of a process which contains other states must be transferred as well. Although a number of techniques are available for transferring the memory space of a process, such as lazy copying [13] and a shared file server [4], at this stage we have limited ourselves to a straightforward implementation – direct copying, because Amoeba does not support virtual memory.

Apart from migrating execution state and memory space of a process, the communication state of the process has to be transferred to the destination. In Amoeba, each thread of the process has its own communication state which remembers the stage of these ongoing RPCs and the role the thread plays (client or server) in those RPCs. These communication states, including the capabilities of its RPC partners, are kept in the kernel.

In order to keep incoming communication "alive" during a process migration, a similar technique to the V system was used in our implementation. Therefore, any messages received for a suspended process are rejected and a "process is migrating" response is returned. The sending machines will retransmit these messages after a suitable delay. The sending machine, by making use of the FLIP protocol, is able to locate the migrated process and send it the message. The same technique allows a reply message to be returned correctly to the client after either client or server has migrated.

In our implementation, when a migration server on the source machine receives a migration request which contains the capability of a process to be migrated, it first suspends the process using a checkpoint function which extracts the process descriptor and sends it to the destination. Later, the process migration server at the destination, based on the segment capabilities contained in the process descriptor, is able to copy the process memory space to its local memory. Finally, the process is resumed at the destination and the source machine removes the original process. Based on this

technique, we avoid a local memory copying of the suspended process from its original place to the memory of the migration server at the source machine.

Note that it is better for the migration server on the destination machine to copy the process memory than for the source migration server to do so. This is because the destination machine normally is under-loaded. In our current implementation, the migration servers are user processes, and Amoeba uses a simple round-robin scheduler which gives all user processes the same priority and the same time quantum (usually 100 ms). Consequently, the migration server which predominately does communication more than computation makes slow progress on a heavily-loaded machine. This could be improved by having a more sophisticated scheduler (the round-robin scheduler is adequate for the processor-pool model) or by making the migration server a kernel process which has priority over user processes.

3 Experimental Study

Performance measurements are the most important part of a load balancing study, which is usually carried out by repeatedly executing some benchmarks under different settings. These benchmarks should produce a workload representative of the system being tested. Unfortunately it is hard to find adequate applications to carry out these measurements because Amoeba, like many other distributed operating systems, is a research-oriented system and there are few applications designed for it. Moreover, few people use Amoeba for production work so that we are unable to collect representative workload traces, which prohibits the possible comparison of performance with/without the load balancing facility under a real workload. In this situation, most researchers use artificial (synthetic) loads to conduct their performance tests. We have taken the same approach at this stage and have developed an artificial workload generator, which can produce a series of processes and send these processes to machines based on the Poisson distribution with tuneable interarrival time and service time. The service time of processes follows a given distribution, e.g. exponential distribution. All the generated processes are independent at this stage: there is no inter-process communication and no execution order is imposed between processes.

3.1 Load and Performance Measurement

Because one of the most important objective of load balancing is to improve the mean response time of

processes, we choose the mean response time to be the measurement standard in our study.

Apart from using the mean response time to determine the quality of algorithms, we need a load index to measure the load level of a computer. Using the available load indices against the criteria listed in [7], we decided to use the number of processes on a machine as the load index in this preliminary study. This is because the number of processes is easy to obtain and it is also easy to determine the threshold dynamically.

3.2 Environment of Experiments

The experiments have been carried out at night using an undergraduate PC laboratory, which has a number of 386s, with 40Mhz, 4MB main memory. All these PCs are connected to an Ethernet segment by Etherlink2 cards. Another PC equipped with 16MB main memory and a 200MB hard disk space is used to run the file server and directory server. Other system servers, such as boot server and time server, are also executed on this machine. We put another server, the statistics server, on this machine to collect the interesting events during an experiment, such as the start time of a process, the terminated time of a process, the migration time, etc. Based on the information collected, the server can produce output for performance evaluation, e.g., the average response time, average load of the system, the average time for each migration, etc.

The Amoeba run server which was originally designed for job initiation has been turned off to avoid interference with our experiments. Instead, we use our own job initiation servers to initialize jobs on computers.

3.3 Load Balancing Algorithms

There are a large number of load balancing algorithms. These algorithms can be classified into different categories [3]. In the first stage, we implemented some simple algorithms which form two classes: non-preemptive and preemptive. In the non-preemptive class, we have a number of job initiation algorithms:

- **Centralized Initiation:** In the centralized approach, there is one load balancer in the whole system which is responsible for assigning new jobs to computers. When a new job arrives, the load balancer relies on its periodically collected load information to determine the machine to execute the new job and starts the job on that machine.

- **Distributed Initiation:** There is a load balancer on each computer, which broadcasts its load to others whenever it detects a load change in its computer. When a new job arrives, the load balancer, based on its own load, the received load information and the age of these information, determines which computer should host the new job and starts the job on that computer. This algorithm tries to start a job on a destination without negotiation. It depends on different selection orders and a stochastic filter to avoid conflict decisions. The filter, based on the age of received load information, G , the number of computers in the system, M , its recent arrival rate, λ , a threshold, C , and a random number r assigned to each computer ($0 \leq r \leq 1$), determines whether the information is obsolete. If there is an underloaded computer for which $r \leq e^{-\frac{G}{M\lambda C}}$, the load balancer starts the new job on that computer. Otherwise, the job is started locally. The aim of using the filter is to reduce the number of RPCs and speed up the decision-making process.

- **Random Initiation:** This is also a distributed method. When a new job arrives at a computer, the local load balancer randomly selects a computer to run the job if it is overloaded at that moment. Otherwise, the job is started locally.

We have also conducted experiments on a number of preemptive load balancing algorithms: one is based on a centralized method, all the others are distributed:

- **Central:** There is one load balancer running in the system, which regularly polls the other computers to obtain system load information. When the load balancer detects load imbalance instances, it selects one of the computers with the lowest load to accept a process from an overloaded machine.
- **Random selection:** Within this algorithm, when a computer becomes overloaded, it randomly selects another computer to shift its load, no matter whether the computer is underloaded or not.
- **Neighbour:** As described in [2], each computer regularly sends request messages to its neighbours, containing its current load information. When the neighbour receives this message and accepts the request, then these two computers become a pair. Later, based on the load difference between them, the computers decide to shift some

load from one to another. After that, the pair is broken.

- **Broadcast:** This is a server-initiated algorithm. Load balancers exchange their load information whenever the load is changed. When a computer becomes underloaded (in our case, when it becomes idle), its load balancer checks whether there are overloaded machines from the received load information. If so, it first selects a computer and a process that is running on that computer, then calls the process migration server to move this process.

3.4 Experimental Results

It is known that non-preemptive load balancing, usually called job placement, introduces much less overhead to a system than preemptive load balancing. On the other hand, one may argue that a preemptive approach is more flexible than a non-preemptive one, especially when the process execution time is not known in advance. A series of experiments have been carried out to evaluate the effectiveness of different job initiation methods. In the following experiments, we fixed the mean service time ($\frac{1}{\mu}$) of processes to 5.0 seconds. In other word, on average each process requires 5 seconds CPU time. By adjusting the mean inter-arrival rate (λ) of the benchmark, we created different workloads, which is $\rho = \frac{\lambda}{\mu}$, for these experiments

Fig. 1 to Fig. 3 show the results of executing the three job initiation methods on a system with 2 computers, 6 computers, and 12 computers, respectively. In these figures, the vertical axis is the ratio of the mean response time of processes to the mean service time of processes; while, the horizontal axis is the average workload on the system. Two additional curves at the top of these figures are for comparison: one is without load balancing; the other is without load balancing but including the overhead of load information exchange. These two curves are only used here to show the cost of exchanging load information. In these experiments, the centralized method outperforms all other methods. With the increase of the number of computers, the centralized job initiation method decreases the average response time of processes. This indicates that the centralized method scales well, with the overhead incurred by communication between the central point and computers more than compensated by the gain received from load balancing. This result also indicates that with more information, the centralized method can make better decisions, in particular,

when the average load exceeds moderate levels.

Random initiation, such as the random policy used by Eager *et al* [5], also can substantially improve system performance even though it does not require any information exchange and starts jobs at different computers based on a uniform distribution. When the number of computers increases, the performance of the random method becomes worse in comparison with the centralized and distributed initiation. This indicates that the random method does not take advantage of the availability of more computers in its destination selection.

The results also show that the centralized initiation method is superior to its distributed counterpart (IB curves in these figures). When the size of a system increases, the difference becomes bigger. This agrees with Zhou and Zicari's claim [16] that a centralized load balancing scheme scales better than a distributed one. With increased load, the performance difference between these two methods also increases. This is for two reasons: 1) in the distributed method, there are a number of decision makers, which may make conflicting decisions; 2) exchanging load information among computers in the distributed methods brings more overhead.

Fig. 4 to Fig. 6 show the effectiveness of preemptive load balancing added to job initiation. All the algorithms, except the central algorithm, which is based on the centralized job initiation, are based on the same distributed initiation method. These figures show that as the number of computers increases, the load balancing algorithms have different reactions. The performance of the centralized algorithm advances further when the number of computers increases. This again indicates that a centralized strategy outperforms its competitors in a system that has 10 – 20 computers. However, comparing Fig. 3 with Fig. 6, one is able to notice the two centralized methods, one based on job initiation, the other based on both job initiation and process migration, only have small differences. This indicates when the size of a system exceeds a threshold (in our case the threshold is 10), job initiation plays the main role in a centralized load balancing scheme, while, process migration only compensates for poor choices by job initiation, such as, assigning long jobs to one computer and short jobs to another. On the other hand, in the case of Fig. 6, the load balancing facility did migrate a number of processes when the load increases, but these process movements contributed little to the performance. This may be partially due to the migration technique used in this research. If a more effective migration technique and

a high speed network are used, the preemptive load balancing should have some differences to the non-preemptive approach, especially when processes execution time is not *a priori* knowledge.

It surprises us that the server-initiated algorithm works reasonably well in these three circumstances. We believe this is due to 1) quick decision making because this algorithm eliminates the negotiation overhead; and 2) a carefully selected filter value which reduces the possibility of using obsolete load information.

Note that the source-initiated algorithm performs reasonable well when there are 6 computers in the system. However, when the number of computers is increased to 12, this algorithm only achieves the same performance as the random-selection algorithm. This phenomenon is now under investigation. At this stage, we think it is because the job initiation used in these experiments overshadows the effects of preemptive load balancing algorithms.

The pair algorithm, introduced by Bryant and Finkel [2], works slightly better than the random algorithm. Because the pair algorithm is based on a time-driven approach to set up pairs and a computer randomly selects its partner, these efforts sometimes are fruitless besides the overhead.

4 Conclusion

In this paper, we described the results of an experimental study of load balancing on the Amoeba system. The result indicates that a centralized load balancing strategy is more effective than a distributed one when there are 10 – 20 computers in a system. This is due to the fact that a centralized approach creates much less overhead than a distributed one. Because the authority to make load balancing decision is concentrated in one place, it avoids possible conflicts between different decision makers, and the negotiation overhead which is used by most distributed algorithms to avoid conflicts.

This research also shows that job initiation is the most important part of a load balancing scheme, especially when the average load of a system is moderate. With the development of heterogeneous distributed systems, using job initiation to attack load imbalance will be the main approach.

This study also indicates that preemptive load balancing can be used to further improve system performance only when the average load of a system exceeds the moderate level. Although this situation is partly

due to the poor performance of the prototype process migration technique used in this study (it takes about 1.5 second to migrate a 100KB process), process migration in general is an expensive operation and should only be used when the migration overhead can be compensated by performance improvement, as Douglass and Ousterhout pointed out [4].

From this study, we also conclude that a distributed operating system needs an effective process scheduling mechanism to support a variety of services and applications. The ability to change scheduling parameters, such as, process priority and time quantum, according to system configuration and applications is important. This is especially important for micro-kernel systems, like Amoeba, because most system servers execute in user mode in such a system. Simple round-robin scheduling is not enough to support services that are time critical. With the approach of very high speed networks (gigabit/s), the delay of process migration will shift from communication delay to processing delay, indicating a need for a tuneable scheduling mechanism which can handle various scheduling principles on demand, for instance, real-time, priority, etc.

The next step in this work is to study the scalability of these algorithms in a system that contains 50 – 100 computers. In addition, communication will be introduced into the benchmark, letting it create jobs with computation bursts plus RPCs. Then, the algorithms used in this paper will be tested again to see the influence of the communication.

References

- [1] A. Barak, A. Shiloh, and R. Wheeler. Flood Prevention in the MOSIX Load-Balancing Scheme. *TCOS Newsletter*, 3(1), 1989.
- [2] R.M. Bryant and R.A. Finkel. A Stable Distributed Scheduling Algorithm. In *Proceedings of the 2nd International Conference on Distributed Computing Systems*, April 1981.
- [3] T.L. Casavant and J.G. Kuhl. A Taxonomy of Scheduling in General-Purpose Distributed Computing Systems. *IEEE Trans. on Software Eng.*, 14(2), Feb. 1988.
- [4] F. Douglass and J. Ousterhout. Transparent Process Migration: Design Alternatives and Sprite Implementation. *Software and Experience*, 1991.
- [5] D.L. Eager, E.D. Lazowska, and J. Zahorjan. Adaptive Load Sharing in Homogeneous Distributed Systems. *IEEE Trans. on Software Eng.*, SE-12(5), 1986.
- [6] D.L. Eager, E.D. Lazowska, and J. Zahorjan. The Limited Performance Benefits of Migrating Active Processes for Load Sharing. *Proc. of the 1988 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*, May 1988.
- [7] D. Ferrari and S. Zhou. An Empirical Investigation of Load Indices for Load Balancing Applications. *Performance '87*, 1987.
- [8] A. Hac. A Distributed Algorithm for Performance Improvement through File Replication, File Migration, and Process Migration. *IEEE Trans. on Software Eng.*, 15(11), 1989.
- [9] P. Krueger and M. Livny. A Comparison of Preemptive and Non-preemptive Load Distributing. *Proc. of the 8th International Conference on Distributed Computer Systems*, June 1988.
- [10] D. Milojicic. *Load Distribution - Implementation for the Mach Microkernel*. Vieweg, 1993.
- [11] C.F. Steketee, W. Zhu, and P. Moseley. Implementation of Process Migration in Amoeba. In *Proceedings of the 14th International Conference of Distributed System*, June 1994.
- [12] A.S. Tanenbaum, R. van Renesse, H. van Staveren, G.J. Sharp, S.J. Mullender, J. Jansen, and G. van Rossum. Experiences with the Amoeba Distributed Operating System. *Communications of the ACM*, 33, December 1990.
- [13] E. Zayas. Attacking the Process Migration Bottleneck. In *Proceedings of the Eleventh ACM Symposium on Operating Systems Principles*, Nov. 1987.
- [14] S. Zhou. A Trace-Driven Simulation Study of Dynamic Load Balancing. *IEEE Trans. on Software Eng.*, Vol.14(9), Sept. 1988.
- [15] S. Zhou, X. Zheng, J. Wang, and P. Delisle. Utopia: a load sharing facility for large, heterogeneous distributed computer systems. *Software-Practice and Experience*, 23(12), 1993.
- [16] S. Zhou and R. Zicari. Object Management in Local Distributed Systems. *The Journal of Systems and Software*, Vol. 8, 1988.

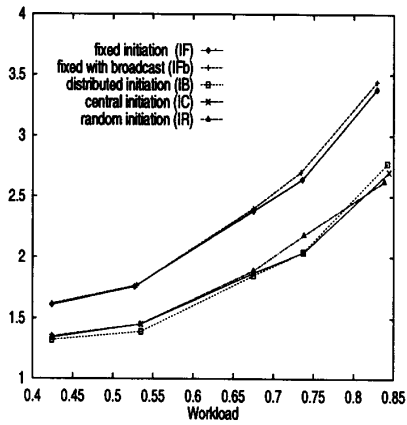


Figure 1: Job initiation on 2 computers

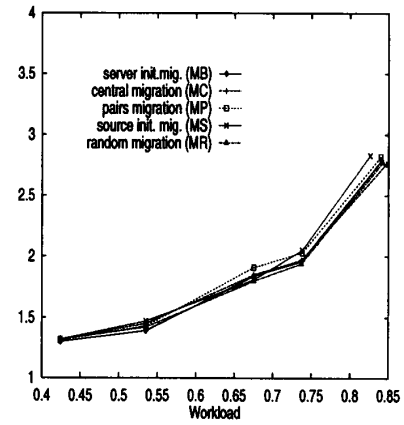


Figure 4: Job initiation plus migration on 2 computers

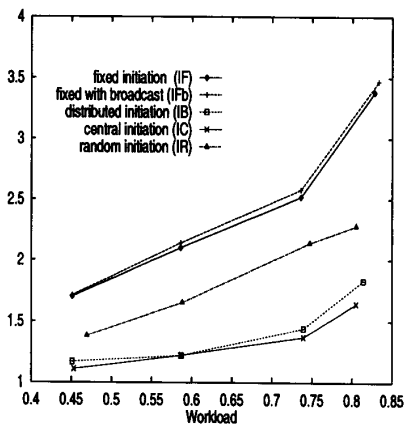


Figure 2: Job initiation on 6 computers

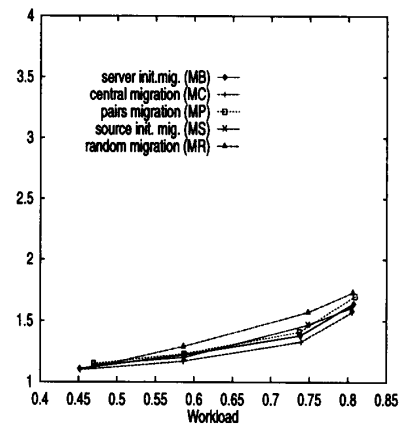


Figure 5: Job initiation plus migration on 6 computers

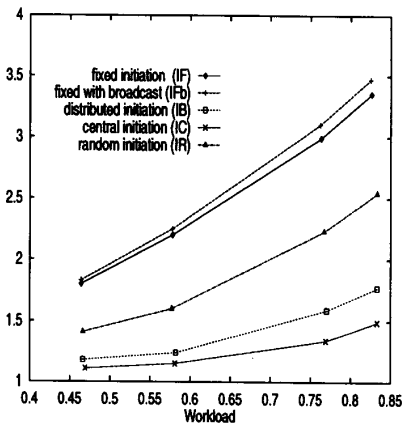


Figure 3: Job initiation on 12 computers

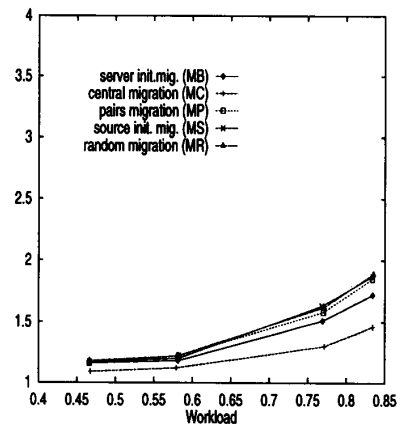


Figure 6: Job initiation plus migration on 12 computers